Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 21, 2026

Abstract

We profile a Micron 2210 512 GB QLC NVMe SSD on a PCIe 3.0 ×4 link with `O_DIRECT` I/O against disposable files, using a `libaio` benchmark cross-checked against FIO. At QD=1, 4 KiB random reads take 53.2 µs (18 783 IOPS) and writes 16.0 µs, because the write buffer hides the flash program. With one request in flight, sequential reads reach only 1 798 MiB/s even at 1 MiB; at QD=32 they peak at 2 144 MiB/s, 57 % of the 3756 MiB/s link, so the NAND back end is the ceiling. Random-read IOPS saturate at QD 64, and Little's Law confirms that the harness keeps the queue full. On a trimmed, idle drive the SLC cache absorbed only 3.8 GiB at 1 071 MiB/s before sequential writes fell to 63 MiB/s. Read p99.9 reaches 5.7× the median at QD=128 (24× after heavy writes), and randomly written data reads 6–9× slower sequentially. Dropping `O_DIRECT` over-reports throughput 19× because it measures the page cache.

1 Introduction

NVMe SSDs deliver high IOPS and throughput only when requests are issued concurrently. Like memory, storage shows a queueing-driven throughput–latency tradeoff: deeper queues raise utilization up to a saturation knee, and past it latency climbs for little extra throughput. We characterize this on a Micron 2210 512 GB QLC NVMe SSD, using `O_DIRECT` I/O against disposable files, a small `libaio` benchmark as the primary engine and FIO as a cross-check (Sections 2 and 3). The report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern, read/write mix and queue depth shape IOPS, MiB/s and latency, and where is the PCIe interface ceiling? (Sections 5 and 6)
3. Where is the queue-depth knee, by what criterion, and what does Little's Law say about the harness? (Section 7)
4. What do tail latency and device state reveal: working-set size, SLC-cache exhaustion and steady state, data layout, and temperature? (Sections 8 and 9)
5. Which setup mistake produces misleading numbers? (Section 10)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: 2 × 32 GiB DDR4-3200, one per channel (A, B), running at 2933 MT/s (dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	a single I/O-issuing thread, not CPU-pinned (storage I/O is device-bound, not CPU-bound, below the host-saturation queue depths of Section 7); no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	Micron 2210 `MTFDHBA512QFD`, 512 GB QLC NVMe [3], firmware `P6MA001` (SMART log taken with root after the run, Section 11)
Interface	PCIe 8.0 GT/s ×4 (Gen3), current and maximum, from sysfs. Raw ceiling $8\,\text{GT/s} \times 4 \times \frac{128}{130} = 3.94\,\text{GB/s}$ (3756 MiB/s; 128b/130b coding [4]), roughly 3.2 GB/s to 3.5 GB/s after TLP/DLLP overhead
Block layer	scheduler `none`, `nr_requests` 1023, 16 I/O queues, volatile write cache “write back”
Target	disposable files on the SSD’s ext4 `/` partition (`nvme0n1p3`, `relatime`, no online discard) via `ACS_SCRATCH`: a 2 GiB test file for the sweeps and a 60 GiB file for the original device-state runs; the lean re-run used a 4 GiB test file (random offsets over its first 2 GiB) and a fill file capped at 40 GiB
Temperature	NVMe composite sensor (`hwmon*/temp1_input`), logged per repetition and every second in the sustained runs
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

2 Methodology and Safety

2.1 Platform

The machine also has a SATA HDD (`/dev/sda`); each run’s `env.txt` resolves the scratch partition to its parent disk and records its `rotational` flag, model, firmware, link and temperature, so a run cannot be silently mislabelled (Section 12 describes how this check once failed).

2.2 Data-loss safeguards

- **Targets.** Only disposable files in a dedicated scratch directory are written, and both are deleted when `run.sh` exits, including on interrupt.
- **Gate.** `scripts/safety.sh` runs before every experiment and refuses block or character devices (also via symlinks), mountpoints, directories, paths outside the scratch directory and targets without enough free space; the scratch directory itself may not be `/`, `\$HOME`, a filesystem root or foreign-owned. It only `stats` paths (Section 3).
- **Benchmark.** `blkbench lstats` its target, refuses anything but a regular file and opens with `O_NOFOLLOW`; no raw device is ever opened.

- **Space.** The large device-state file is refused unless 10 GiB stay free.

2.3 I/O engines and configuration

The primary engine, `build/blkbench`, is a small O_DIRECT `libaio` benchmark (the engine FIO also uses [1]). It keeps exactly QD requests in flight in a closed loop and records every I/O's submit-to-completion latency. FIO 3.42 runs the same points through `fio/job.fio` (Section 3), and `ENGINE=fio` routes every sweep through FIO.

Engine	`libaio`; `O_DIRECT`; 4 KiB-aligned buffers and offsets; 64-bit uniform random offsets (splitmix64)
Timing	4 s per repetition (6 s for tail); write points also stop after 256 MiB (128 MiB and a 15 s pause in the lean re-run); preconditioning absorbs warm-up
Repetitions	5 per point (3 for block-size writes, 4 per engine or span in parts of the lean re-run); median with inter-quartile-range (IQR) bars
Data pattern	incompressible xorshift64* buffers, fixed per queue slot (we did not test whether the 2210 deduplicates)
Latency metric	per-I/O submit→completion; p50/p95/p99/p99.9 from the full sorted sample
Preconditioning	sequential O_DIRECT write over the test file before every sequential-read group (Section 9.3). In the reported `all` run the file was otherwise preconditioned only before `zero_queue` (the current `run.sh` does it before every experiment); the lean re-run did it once, before `tail`
Driver	`SSD_Performance_Profiling/scripts/run.sh`; `src/blkbench.c`

2.4 Execution, run order and idle periods

The author ran all timed runs in a terminal (governor `performance`, turbo and SMT off). `run.sh all` ran in the order of Table 2 on 2026-10-21, 10:32–10:54, followed directly by `SUSTAIN_GIB=60 run.sh device_state` until 11:23 (run `20261021-103201`); an earlier pilot was discarded. Nothing in between trimmed or idled the drive, so each experiment inherited the SLC-cache fill and background work of the previous one, which shaped the write collapses in Sections 5 and 7, the tails in Section 8 and Section 9.

Idle period. An *idle period* means that the drive received no I/O from our scripts for a stated time (5 min to 30 min), so that the controller could finish background SLC-to-QLC folding and garbage collection; an *idle drive* is one measured after such a period.

Lean re-run. The confounded experiments were re-measured that evening with `rerun_clean.sh lean` (run `20261021-220614`, 22:06–22:32 EDT; Table 3) after `sudo fstrim -v /` at 21:34:54 and an idle period of at least 30 min, with the sustained fill first and stopped soon after the cliff. Lean points replace only the matching configurations of the original run; captions name each point's run.

3 Validation and Sanity Checks

`make -C SSD_Performance_Profiling test` (`scripts/validate.sh`) checks that the benchmark measures the SSD and not an artifact. It writes one 64 MiB scratch file and deletes it.

- **Safety gate.** `safety.sh` refuses both block devices, a symlink to `/dev/sda` inside the scratch directory, a directory, a path outside it (`/etc/passwd`), a mountpoint (`/`), and scratch directories at `/`, `\$HOME`, a filesystem root (`/dev/shm`) or not owned by the user (`/usr/share`); each test checks that the refusal names the intended rule. `blkbench` independently refuses a directory and a symlink.

Table 2: Measured wall-clock time of each experiment in the original 2026-10-21 runs, in execution order, and the maximum bytes it can write with the default knobs (byte caps plus preconditioning).

Experiment	Time	Max written	Notes
`zero_queue`	1.1 min	7 GiB	includes the initial preconditioning
`block_size`	9.1 min	22 GiB	48 points; write points 3 reps, 256 MiB cap
`rw_mix`	0.5 min	4 GiB	
`queue_depth`	3.7 min	11 GiB	read and write, QD 1–256
`tail`	2.0 min	–	reads only
`pitfall`	0.8 min	2 GiB	reads only (plus re-fill)
`layout`	2.2 min	6 GiB	includes a 4 GiB random overwrite
`fio_check`	3.3 min	8 GiB	both engines, interleaved
`sustained`	24.9 min	72 GiB	60 GiB fill + ≤8 GiB random + ≤4 GiB recovery
`working_set`	4.0 min	5 GiB	on the 60 GiB file left by `sustained`

Table 3: The lean re-run (`rerun_clean.sh lean`, 2026-10-21, 22:06–22:32 EDT) in execution order: wall-clock time from the run log, including idle periods, and the most each step could write.

Step	Time	Max written	What it re-measured
`sustained`	1.6 min	6.3 GiB	fill (stopped at 4.3 GiB), 1 GiB random, 60 s idle, 1 GiB recovery
`tail`	12.1 min	4 GiB	10 min idle, fresh 4 GiB precondition, QD 8/32/64/128 reads
`working_set`	1.6 min	0.75 GiB	spans 1, 2, 4 GiB, forward then reverse
`block_size`	8.5 min	1.1 GiB	QD=32 random reads (8 sizes), 5 min idle, QD=1 random writes ≥256 KiB
`queue_depth`	1.1 min	2.5 GiB	random writes at QD 4–32
`fio_check`	1.0 min	3 GiB	the three write points, engine order alternating

- **Incompressible data.** The preconditioned file does not shrink under `gzip -1`.
- **Physical orderings.** 128 KiB sequential beats 4 KiB random throughput, and QD 1→8 raises $L = \lambda W$ without exceeding QD (Section 7).
- **Direct I/O is real.** The `-buffered` path reaches 19× the sequential and 71× the random `O_DIRECT` IOPS (Table 11), so the real runs bypass the page cache; `make test` asserts a gap of more than 5×.
- **Interface ceiling and identity.** No result may exceed the 3756 MiB/s link (the highest, 2 144 MiB/s, does not), and every `env.txt` records the parent disk, `rotational`=0, model, firmware and link.

`iostat` and `perf` are unavailable, so effective queue depth is checked in-band (Table 8).

3.1 Cross-check against FIO

`run.sh fio_check` runs seven points (QD=1 and 32; reads and writes; 4 KiB random and 128 KiB sequential) through both engines, alternating them repetition by repetition. FIO's `lat` percentiles measure the same submit→completion time (log-bucketed, ∼3 % bin error). We treat agreement within 10 % in IOPS and p50 as validation (Table 4).

Reads agree: IOPS ratios of 0.983–1.006 and p50 within 1 %. (The two 128 KiB sequential points ran at only ∼225 MiB/s in both engines because `fio_check` followed `layout` without a re-fill, since fixed; they measured the fragmented layout of Section 9.3.)

Table 4: `blkbench` vs. FIO on identical points (medians of interleaved repetitions: 5 per engine for reads, 4 per engine for writes, 2 of them in each position; latencies in µs). Data: the write points from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

Pattern	Block	QD	IOPS		FIO/blk	p50		p99	
			blk	FIO		blk	FIO	blk	FIO
rand read	4 KiB	1	18 763	18 438	0.983	49.9	50.4	58.7	57.6
rand read	4 KiB	32	129 580	129 705	1.001	217.1	216.1	663.5	692.2
seq read[†]	128 KiB	1	1 793	1 804	1.006	546.4	544.8	1 057.9	1 036.3
seq read[†]	128 KiB	32	1 827	1 827	1.000	17 401.3	17 432.6	19 449.4	19 791.9
rand write	4 KiB	1	66 643	59 109	0.887	14.7	15.3	16.9	17.7
rand write	4 KiB	32	226 524	254 031	1.121	121.7	119.8	150.3	154.6
seq write	128 KiB	1	8 388	8 303	0.990	99.4	98.3	259.9	286.7

[†] Grey rows: fragmented layout, not representative of sequential-read performance (the file had just been randomly overwritten; see text). They still compare the engines.

Writes vary more, mostly with drive state. The lean-run write points give ratios of 0.990 (sequential), 0.887 and 1.121 (random, QD=1 and 32), with p50 within 1 µs. In Table 5, position within a pair moves the sequential write from 9 146 to 7 723 IOPS for the same engine, more than the engine gap, and the QD=32 random-write ratio flipped from 0.78 in the original run, which a fixed engine bias would not do. FIO's consistently slower QD=1 random write (also 0.93 originally) may reflect a small per-I/O host cost. We give write numbers from either engine a ∼15 % uncertainty.

Table 5: Write points of `fio_check` split by engine and by position within each pair of repetitions: median IOPS of 2 repetitions per cell (data: lean re-run of 2026-10-21).

Point	blkbench 1st	blkbench 2nd	FIO 1st	FIO 2nd
4 KiB rand, QD=1	65 641	66 758	57 845	61 637
4 KiB rand, QD=32	226 524	232 289	239 456	254 031
128 KiB seq, QD=1	9 146	7 723	8 755	7 938

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

At QD=1 a single I/O is in flight, so latency is the full host-stack + controller + flash path. We expect 4 KiB random reads in the tens of microseconds (flash read dominates) and 128 KiB sequential transfers to be bandwidth-limited at a few hundred µs. Writes should be *faster* than reads at QD=1 because the SSD's DRAM/SLC write buffer acknowledges them before the flash program completes.

Fixed	iodepth 1; one job; `O_DIRECT`; 5 repetitions of 4 s (write points also stop at 256 MiB)
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Order	random read → random write → sequential re-fill of the file → sequential read → sequential write (the re-fill keeps the sequential read independent of the random writes; Section 9.3)
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

Table 6: QD=1 performance on the Micron 2210 NVMe SSD: medians over repetitions, with the IQR in brackets for IOPS and p99; latencies in µs. Random reads pay the full flash-read latency; random writes are faster because the write buffer absorbs them.

Pattern	Block	IOPS [IQR]	MiB/s	Mean	p50	p95	p99 [IQR]	p99.9
rand read	4 KiB	18 783 [18 767, 18 796]	73.4	53.2	49.9	50.9	62 [62, 71]	534
rand write	4 KiB	62 217 [62 067, 62 239]	243.0	16.0	14.6	15.9	18 [18, 18]	490
seq read	128 KiB	11 530 [11 511, 11 543]	1 441.2	86.7	80.5	114.2	491 [490, 496]	563
seq write	128 KiB	5 100 [891, 5 298]	637.5	196.0	90.0	1 156.4	1 437 [1 436, 34 765]	9 853

4.1 Analysis

The QD=1 numbers identify an NVMe SSD, not a rotational disk: 4 KiB random reads take 53.2 µs, dominated by the NAND read plus the NVMe/`libaio` host path, with no seek. Random *writes* take only 16.0 µs (62 217 IOPS), because the controller acknowledges them from its write buffer (SLC cache plus SRAM) before the flash program finishes; a read has no such buffer. The 128 KiB sequential read takes 87 µs per request (1 441 MiB/s): one outstanding request leaves the device idle between transfers, which Sections 5 and 7 remove. The p99.9 column already shows background flush and GC outliers (Section 8).

The sequential-write row is unstable (IOPS IQR 891–5 298). Its first two repetitions ran at 490 and 891 IOPS with p99 of 30 ms to 35 ms, the last three at 5100 to 5460 IOPS, with p50 near 90 µs throughout. It directly followed the 2 GiB sequential re-fill, so we read the slow repetitions as the controller flushing that burst; the median describes the recovered state, not a stable baseline. The re-fill itself exists because a discarded pilot that read sequentially right after the random writes got only a fraction of this rate (Section 9.3).

5 Block-Size Sweep

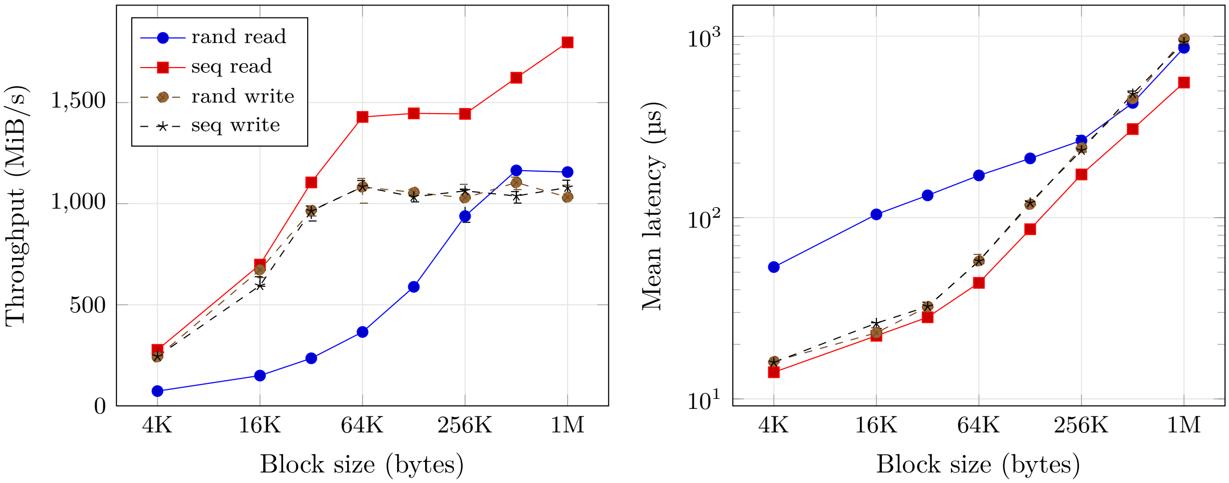
Hypothesis (from first principles, then compared with the data)

As the block grows from 4 KiB to 256 KiB at QD=1, small-block I/O should be *IOPS-limited* (throughput = IOPS × block, rising with block size) and large-block I/O should approach a *throughput ceiling* where IOPS falls as 1/block. Sequential should beat random at every block size, with the gap largest at small blocks where per-op overhead dominates.

Fixed	one job; `O_DIRECT`; 5 repetitions of 4 s for reads; 3 repetitions capped at 256 MiB each for writes
Varied	block size 4 KiB–1 MiB (8 sizes); random vs. sequential; 100% read vs. 100% write; QD=1 (all four combinations) and QD=32 (sequential read and write)
Ceiling	PCIe 3.0 ×4: 3756 MiB/s raw (Section 2)
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`

5.1 Analysis

Random reads at QD=1 are latency-bound, not size-bound. From 4 KiB to 256 KiB, IOPS fall only from 18 711 to 3 754, so throughput climbs from 73 to 939 MiB/s while latency grows only 5× for a 64× larger block. We read this as $W(b) \approx W_0 + t_{\text{xfer}}(b)$: a fixed cost $W_0 \approx 50\,\mu\text{s}$ (NAND sense plus command overhead) plus a transfer term that grows sub-linearly while the controller



(a) QD=1 throughput.

(b) QD=1 mean latency (same runs).

Figure 1: QD=1 block-size sweep, reads (solid) and writes (dashed); medians with IQR bars. Random-read throughput rises with block size because latency grows more slowly than the transfer size. Sequential reads stay near 1.4 GiB/s from 64 to 256 KiB and reach 1.8 GiB/s at 1 MiB, a single-request limit well below the 3756 MiB/s link. Random and sequential writes both stay at the ~1.05 GiB/s SLC ingest rate from 64 KiB up; the original run's collapse of the ≥256 KiB random writes did not recur after an idle period (see text). Data: the random writes ≥256 KiB from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

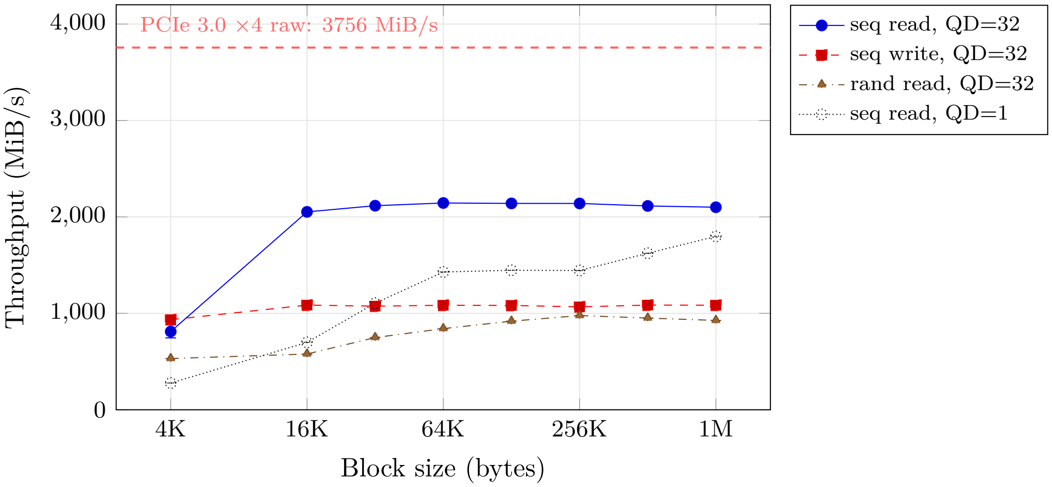


Figure 2: Sequential throughput at QD=32 against the measured-link ceiling (red), with the QD=1 read curve for reference. Deep queues remove the per-request latency limit, so the remaining gap to the line is the device's internal bandwidth plus protocol overhead. Data: the QD=32 random reads from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

stripes a request across dies. Beyond 256 KiB latency doubles with size and throughput plateaus at 1 164–1 156 MiB/s.

Sequential reads at QD=1 hit a single-request limit. Read-ahead cuts the 4 KiB sequential read to 14.0 µs (random: 53.4). From 64 to 256 KiB throughput is flat at 1 429–1 447 MiB/s, reaching 1 798 MiB/s at 1 MiB, only 48 % of the link: the limit is how fast the controller streams one command.

Deep queues do not reach the link ceiling (Figure 2). At QD=32 sequential reads flatten at 2 052–2 144 MiB/s (2 100 MiB/s at 1 MiB), about 56 % of the raw link, well below the 85 % to 90 % protocol overhead allows. The NAND/controller back end is the limit, consistent with Micron's 2200 MB/s read and 1070 MB/s write rating (≈2098 and 1020 MiB/s) [3]. QD=32 sequential writes are flat at 1 084 MiB/s, the SLC ingest rate (Section 9).

Random reads at QD=32 (lean re-run) reach 135 967 IOPS at 4 KiB, peak at 977 MiB/s and give 927 MiB/s at 1 MiB, against 2 100 for sequential: random placement costs more than half the bandwidth even with 32 large requests in flight.

Writes at QD=1 post 61 897 IOPS at 4 KiB (reads: 18 711), and random and sequential curves coincide (1 083 vs. 1 084 MiB/s at 64 KiB), flattening at the same ∼1.05 GiB/s ingest rate as at QD=32: the buffer acknowledges before the NAND program, so writes reach their ceiling with one request in flight.

A drive-state episode, not a block-size effect. In the original run, after about 10 GiB of writes, the random writes at 256 KiB, 512 KiB and 1 MiB fell to the direct-to-QLC rate (64 MiB/s to 66 MiB/s, p99 33 ms to 45 ms). Re-measured after a 5 min idle period with 15 s pauses, they ran at 1 029, 1 104 and 1 032 MiB/s, on the sequential-write curve: the collapse was cache exhaustion.

6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

At a fixed block size and queue depth, a mixed read/write workload should deliver *fewer* total IOPS than either pure workload, not a weighted average: reads and writes contend for the controller and flash channels, and a read queued behind a slow program/erase inherits its latency. We expect pure writes to post the highest IOPS at QD=16 (write-buffer absorption) and the 50/50 mix to be the worst.

Fixed	4 KiB random; iodepth 16; one job; `O_DIRECT`; 5 repetitions of 4 s (points with writes also stop at 256 MiB)
Varied	reads per 10 I/Os, deterministically interleaved: 10 (100% R), 7 (70/30), 5 (50/50), 0 (100% W)
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

Table 7: Read/write mix at QD=16, 4 KiB random: medians with the IQR in brackets; latencies in µs.

Mix	Total IOPS	Mean latency	p99
100% read	102 514 [102 472, 102 633]	154.1 [153.9, 154.2]	556 [555, 561]
70/30	81 497 [79 349, 84 712]	193.7 [186.4, 199.3]	757 [718, 796]
50/50	69 172 [68 792, 72 950]	229.4 [217.3, 230.8]	989 [962, 1 008]
100% write	253 874 [216 774, 254 678]	62.7 [62.5, 72.8]	79 [69, 528]

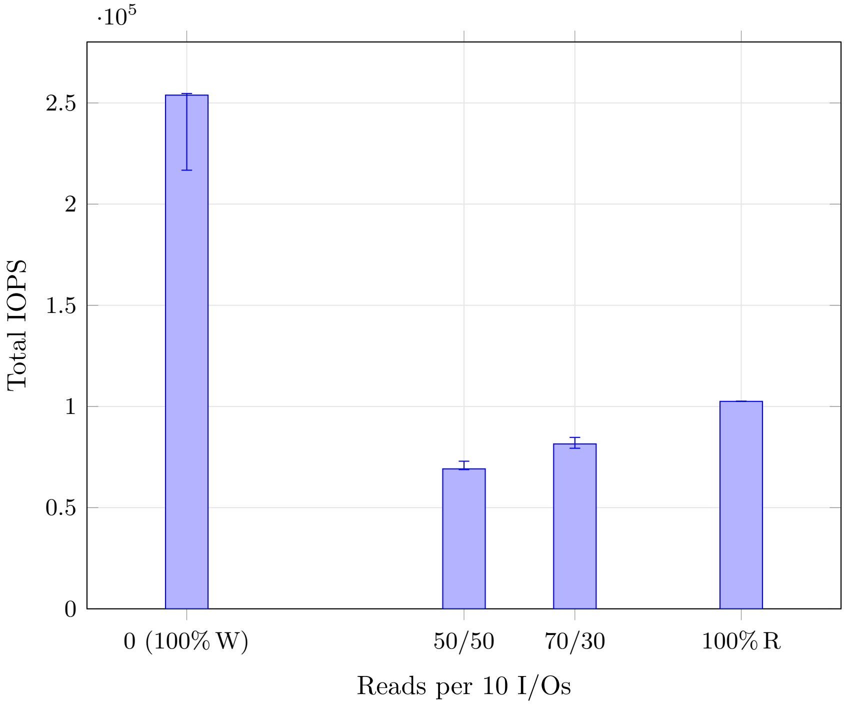


Figure 3: Total 4 KiB random IOPS at QD=16 across read/write mixes (median, IQR bars). Both pure workloads beat the mixes; the 50/50 mix is the worst.

6.1 Analysis

Pure writes are fastest, at 253 874 IOPS and 63 µs mean, because the controller absorbs them into its SLC/SRAM buffer; pure reads reach 102 514 IOPS. Mixing is worse than either: 70/30 drops to 81 497 IOPS and 50/50 to 69 172, below even pure reads. A write occupies a die (program, or the buffer flush it triggers), reads mapped to that die queue behind it, and interleaving breaks up the buffer's batching.

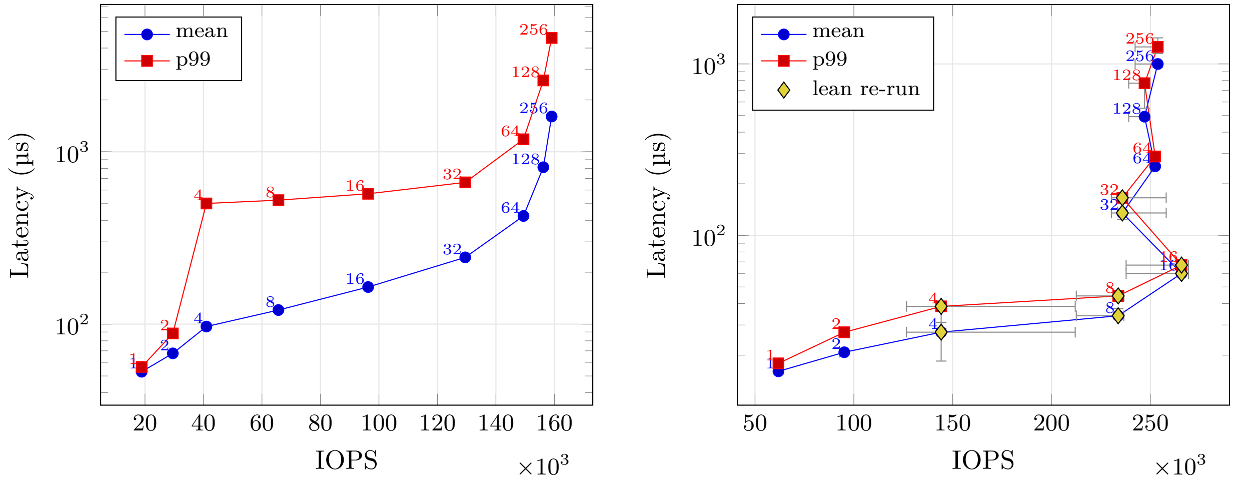
p99 rises from 556 µs for pure reads to 757 (70/30) and 989 µs (50/50), but pure writes have the lowest median p99 and by far the widest IQR (69–528 µs): a repetition either stays inside the buffer or meets a flush. A "70% read" workload is thus not a weighted average of the pure results; mixes must be measured, not interpolated.

7 Queue-Depth Sweep

Hypothesis (from first principles, then compared with the data)

IOPS should rise steeply with queue depth as the SSD's parallel NAND channels fill, then saturate at a knee beyond which latency grows roughly linearly with queue depth for no extra throughput. By Little's Law the number of outstanding I/Os $L = \lambda W$ should track the configured iodepth while the queue stays full; where it stops tracking reveals a host-side or engine limit.

Fixed	4 KiB random; one job; `O_DIRECT`; 5 repetitions of 4 s (writes also stop at 256 MiB)
Varied	iodepth 1, 2, 4, . . . , 256; 100% read and 100% write
Knee rule	the smallest QD whose doubling raises median IOPS by less than 10 % (computed by `run.sh` into `queue_depth_knee.csv`)
Effective QD	checked in-band with Little's Law (Table 8); no `iostat` available
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`



(a) 4 KiB random read.

(b) 4 KiB random write.

Figure 4: Throughput vs. latency as queue depth grows; each point is labelled with its QD. Bars show the IQR across repetitions in both IOPS and latency. Read IOPS reach 149 422 at the knee (QD 64) and at most 159 047 at QD=256; deeper queues then add only latency. In (b), diamonds mark the write points at QD 4–32, which come from the lean re-run; the other write points (QD 1, 2, 64–256) come from the original run. Data: the write points at QD 4–32 from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

7.1 Knee

We define the knee as *the smallest QD whose doubling raises median IOPS by less than* 10 %: past it, most extra concurrency becomes latency rather than throughput.

Reads. The rule gives QD 64 (149 422 IOPS, 424 µs mean). Doubling to QD 128 adds only 4.5 % IOPS, while QD 32→64 still gained 15 %, so the knee lies between 32 and 64.

Writes. QD 4–32 come from the lean re-run (diamonds in Figure 4b), the rest from the original run. The rule gives QD 16 (265 754 IOPS, 59.9 µs mean); doubling to QD 32 changes IOPS by −11.3 %. IOPS rise from 61 860 at QD=1 to the QD=16 peak and stay at 235 830–253 743 beyond; the QD=32 dip is within the overlapping IQRs (230 331–257 997 vs. 237 723–269 281). In the original run a flush stall at QD=8 (16 884 IOPS, p99 27.9 ms) made the rule return QD=4; it did not recur after the lean pauses. The write knee is only indicative, since the curve joins two drive states. The plateau of ∼250k IOPS × 4 KiB ≈ 0.97 GiB/s is the SLC ingest rate: writes saturate the buffer's intake, not the NAND parallelism that limits reads.

7.2 Little's Law as a consistency check

Little's Law [2] states $L = \lambda W$ for the mean number in system L, throughput λ and mean time in system W. In a closed loop it holds by construction: $\lambda W = \mathrm{QD} \cdot W/(W + g)$, where g is the host gap between reaping and resubmitting. So $L/\mathrm{QD} \approx 1$ shows that the timers are consistent and the host keeps up ($g \ll W$), while a clearly lower ratio would expose a host bottleneck. As a control, `make test` on tmpfs, where I/O completes inside `io_submit`, measured $L/\mathrm{QD} \approx 0.57$ ($L = 4.56$ at QD=8; `data/raw/validate_tmpfs/20261021-231433.txt`), which the harness flags. L includes time in the kernel block layer, so device-queue occupancy can only be lower.

Table 8: Little's Law per queue depth: λ (IOPS) and W (mean latency, µs) are medians with the IQR in brackets; $L = \lambda W$ is computed per repetition and then summarized. `4 KiB` random.

Mode	QD	λ (IOPS) [IQR]	W [IQR]	$L = \lambda W$ [IQR]	L/QD
read	1	18 866 [18 854, 18 885]	52.9 [52.9, 53.0]	1.00 [1.00, 1.00]	0.998
read	2	29 518 [29 496, 29 525]	67.6 [67.6, 67.7]	2.00 [2.00, 2.00]	0.998
read	4	41 006 [40 991, 41 072]	96.8 [96.6, 96.8]	3.97 [3.97, 3.97]	0.992
read	8	65 629 [64 935, 65 709]	120.7 [120.5, 122.0]	7.92 [7.92, 7.92]	0.99
read	16	96 303 [96 269, 96 406]	164.1 [163.9, 164.2]	15.80 [15.80, 15.80]	0.988
read	32	129 513 [129 050, 129 607]	243.9 [243.8, 244.4]	31.59 [31.54, 31.60]	0.987
read	64	149 422 [149 292, 149 458]	424.2 [424.2, 424.5]	63.38 [63.38, 63.39]	0.99
read	128	156 211 [156 149, 156 431]	813.4 [812.2, 813.6]	127.05 [127.04, 127.06]	0.993
read	256	159 047 [158 980, 159 094]	1 602.6 [1 602.2, 1 603.3]	254.89 [254.89, 254.89]	0.996
write	1	61 860 [61 800, 62 223]	16.1 [16.0, 16.1]	1.00 [1.00, 1.00]	0.996
write	2	95 161 [92 914, 95 226]	20.8 [20.8, 21.3]	1.98 [1.98, 1.98]	0.989
write	4	144 161 [126 613, 211 985]	27.2 [18.5, 31.1]	3.93 [3.92, 3.94]	0.982
write	8	233 722 [212 524, 236 579]	34.0 [33.6, 37.4]	7.95 [7.94, 7.95]	0.993
write	16	265 754 [237 723, 269 281]	59.9 [59.1, 67.0]	15.92 [15.92, 15.93]	0.995
write	32	235 830 [230 331, 257 997]	135.1 [123.6, 138.4]	31.88 [31.87, 31.88]	0.996
write	64	252 431 [250 115, 252 712]	252.8 [252.6, 255.2]	63.83 [63.80, 63.83]	0.997
write	128	247 061 [239 034, 247 254]	492.5 [491.5, 502.3]	121.77 [121.44, 127.54]	0.951
write	256	253 743 [242 276, 255 085]	999.1 [956.0, 1 003.7]	254.67 [254.67, 254.86]	0.995

Inference: internal parallelism. Applied to the device, with $\lambda_{\max} \approx 159\,047$ reads/s and an assumed service time S between 40 µs and the QD=1 latency (53 µs), only $L_{\text{busy}} = \lambda_{\max} S \approx 6$–$8$ reads are in service at once; of the 64–256 outstanding past the knee, the rest wait in queues. With 1-Tbit dies (4 in a 512 GB drive; Micron does not publish this) that is about two reads per die, a plausibility argument only.

7.3 Analysis

Read IOPS climb from 18 866 at QD=1 to 96 303 at QD=16 and 129 513 at QD=32, then flatten at 149 422–159 047. Scaling is sub-linear from the start. Beyond the knee mean latency doubles with each doubling of QD (424→813→1 603 µs), the textbook saturation signature: with λ flat, every added request simply waits.

For reads L/QD stays at 0.987–0.998 up to QD=256 (Table 8), so the single `libaio` thread keeps the queue full and the saturation is the device's. For writes it is 0.95–0.997; the low 0.95 at QD=128 has an L IQR reaching 0.996 of QD (121.4–127.5), so we cannot tell a stall from noise.

In practice, reads need QD ≈ 32–64 for nearly all the throughput at a quarter to half a millisecond; deeper queues only add latency. Writes need only QD ≈ 16 while the SLC cache has room; once it is full, QLC programming sets the rate at any depth (Section 9).

8 Tail Latency

Hypothesis (from first principles, then compared with the data)

Moving from a mid queue depth to the near-knee depth should raise the median modestly but stretch the upper percentiles much more: deeper queues mean more requests waiting behind the occasional slow flash operation, so the p99/p99.9 gap over p50 should widen.

Fixed	4 KiB random read; one job; 6 s runs, 5 repetitions; NVMe temperature recorded per repetition
Varied	iodepth 8 (mid), 32 and 64 (bracketing the knee of Section 7), 128 (past it)
Drive state	lean re-run: after the short sustained fill, the large file was deleted, the drive was idle for 10 min, and a fresh 4 GiB test file was preconditioned sequentially; reads cover its first 2 GiB
Script	`SSD_Performance_Profiling/scripts/run.sh tail` (inside `rerun_clean.sh lean`)

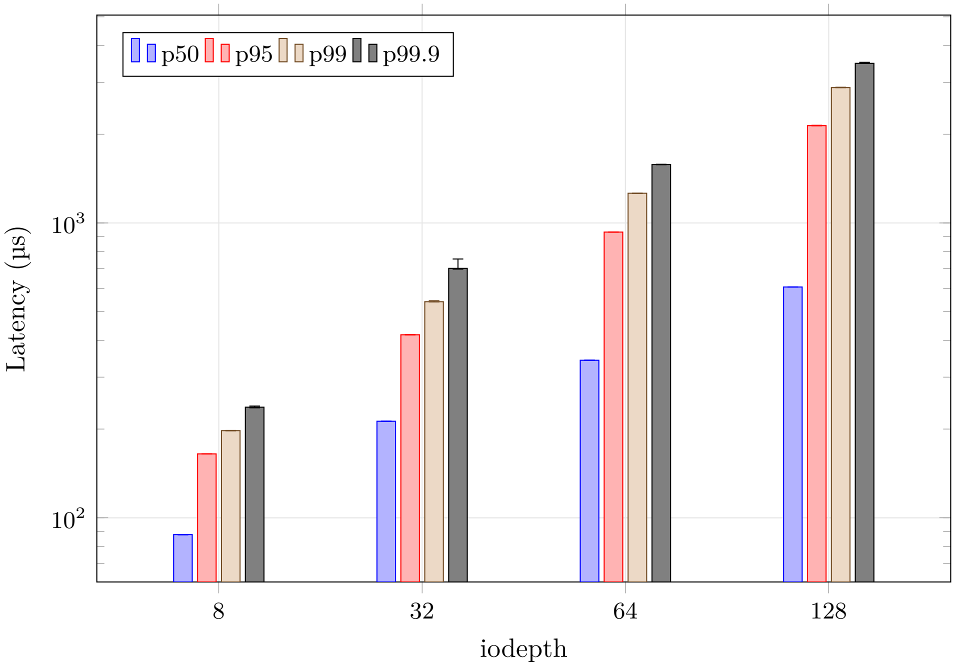


Figure 5: Latency percentiles at a mid queue depth (8), around the knee (32, 64) and past it (128); medians with IQR bars (data: lean re-run of 2026-10-21). Measured after a 10 min idle period on a freshly preconditioned file. The upper tail widens faster than the median as QD grows: p99.9/p50 rises from 2.7× at QD=8 to 5.7× at QD=128.

8.1 Analysis

On an idle drive the tail widens with queue depth as hypothesised, but modestly. QD=8: p50 88, p99 198, p99.9 237 µs (p99.9/p50 = 2.7×); QD=32: 213, 541, 701 µs (3.3×); QD=64: p50 342, p99.9 1 578 µs (4.6×); QD=128: p50 607, p99.9 3 478 µs (5.7×). From QD=8 to 32 the median grows 2.4× and p99.9 3.0×; from 32 to 64, 1.6× and 2.3×, while IOPS barely grow past the knee (152 073→157 257). Every added request waits, and one queued behind an occasional slow read waits longer, so the upper percentiles stretch more than the median; p99/p50 follows the same trend (2.3× to 4.7×). IOPS match the original queue-depth sweep at QD=32 (135 794 vs. 129 513) and QD=128 (157 257 vs. 156 211). At QD=8 the idle drive is faster (80 005 vs. 65 629 IOPS) with a much lower p99 (198 vs. 524 µs); we suspect, without testing, that the original sweep met background work from earlier writes.

The tail depends on recent writes. The original tail run (`tail/20261021-103201`) started right after ∼11 GiB of random writes with no idle period. At QD=32 it delivered 64 475 IOPS, half the idle-drive value, at a median of 385 µs, and its p99.9/p50 rose from 4.1× at QD=8 to 24× at QD=128 (p99.9 16.0 ms). It was no hotter (50.9 °C to 52.9 °C vs. 39.9 °C to 52.9 °C), and although file size and time of day also differed, the likely cause is reads waiting behind background folding and garbage collection. Queueing alone keeps the tail ratio at 3–6×; recent heavy writes push it to 24×. For a latency SLA these percentiles matter more than the mean: at QD=32 one read in a thousand takes 701 µs on an idle drive but took 2.6 ms after heavy writes, and past the knee these tails inflate fastest for almost no IOPS.

9 Device State and Working Set

Hypothesis (from first principles, then compared with the data)

Back-to-back sustained sequential writes should first ride the SLC/DRAM write cache (a burst), then settle to a lower steady-state throughput once the cache must flush to QLC in the background. Occasional latency spikes from garbage collection should appear in the upper tail even while mean throughput is steady.

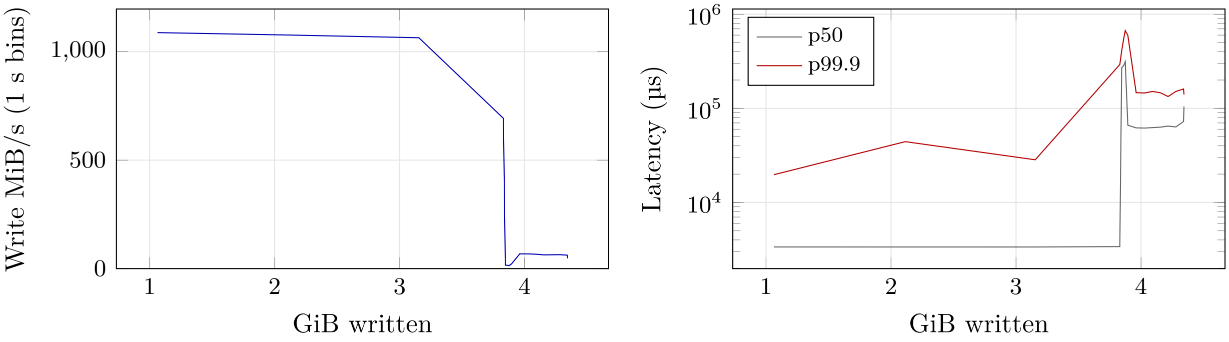
Hypothesis (from first principles, then compared with the data)

(Added before the large-file runs.) (i) A single sequential write far larger than the SLC cache should show a sharp *cliff*: burst throughput near the QD=32 write rate of Section 5, then a drop to the native QLC program rate, several times lower. (ii) After that, random 4 KiB writes reach a lower steady state than in the burst, with heavier tails from garbage collection. (iii) A short idle period lets the controller drain the cache and recover part of the burst rate. (iv) Random reads should slow as the working set grows from 1 GiB to 60 GiB, because more logical-to-physical map lookups miss the controller's mapping cache. (v) "Sequential" reads of data written randomly should be much slower than of data written sequentially. (vi) Temperature should rise during sustained writes but stay below the 86.85 °C warning threshold reported in sysfs (`temp1_max`).

`sustained`	lean re-run: sequential 128 KiB QD32 fill of a fresh file (at most 40 GiB), stopped 10 s after throughput stays below 30 % of the burst; then 4 KiB QD32 random writes (60 s or 1 GiB), a 60 s idle gap and a 1 GiB recovery probe; logged every second
`working_set`	lean re-run, fresh 4 GiB file after `tail`: 4 KiB random read (QD1, QD32) and write (QD32) over its first 1, 2, 4 GiB, then in reverse order, 2 repetitions per pass
`layout`	original run, 2 GiB file: 128 KiB sequential (QD1, QD32) and 4 KiB random reads after a sequential fill, then after a 4 GiB random overwrite (∼86 % of blocks)
Drive state	`fstrim` at 21:34:54, then at least 30 min idle; fill at 22:06:25. 77 GiB free on the scratch partition; other partitions (one untouched 351 GiB) not trimmed
Script	`rerun_clean.sh lean`; `run.sh layout` (inside `all`)

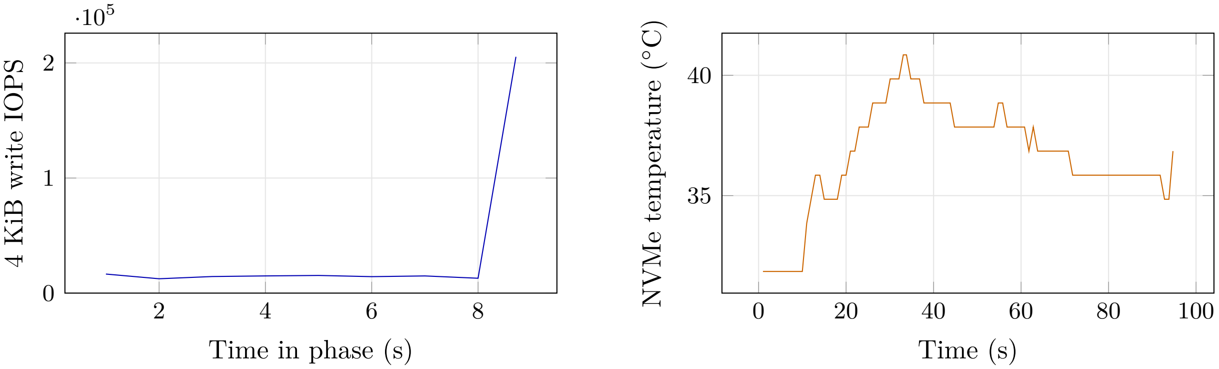
9.1 SLC-cache exhaustion and steady state

Analysis. The fill ran at the SLC ingest rate, 1 071 MiB/s, for 4 s and collapsed after 3.8 GiB: four seconds at 13 MiB/s to 21 MiB/s (median latency up to 0.31 s), then seven at 62 MiB/s to 68 MiB/s. The post-cliff median, 63 MiB/s (0.059 of the burst, a 17× drop, far more than the "several times" of



(a) Sequential fill: throughput vs. volume.

(b) Sequential fill: per-second latency.



(c) Random-write phase (QD32).

(d) Composite temperature, all phases.

Figure 6: Sustained-write experiment, one-second samples (data: lean re-run of 2026-10-21). (a) The SLC-cache cliff appears as the drop in throughput after 3.8 GiB; the stop rule ended the fill at 4.3 GiB. (b) The latency during the fill. (c) The random-write phase after the fill; its last, partial sample (0.7 s) jumped to SLC speed and reached the 1 GiB cap. (d) Temperature across the baseline idle, fill, random, idle and recovery phases.

Table 9: Sustained-write summary (computed by `run.sh` into `sustained_summary.csv`). Burst is the median of the leading seconds that stay above half the first second's rate. The cliff is the first second, starting a 5-second window, whose rate falls below half the burst; the table gives the volume written before it (data: lean re-run of 2026-10-21). Steady is the median of all seconds after the cliff. The lean re-run stops the fill 10 seconds after the cliff; without a cliff the fill volume would be a lower bound on the cache size. The random phase mixes two states, so the table gives its byte-weighted average and the per-second quartiles and median p99 over all 9 samples, including the final partial (0.7 s) one; the text quotes the eight full seconds. The recovery probe reached its 1 GiB cap within its first second, so its first and last 5 s are the same sample.

Burst sequential write (MiB/s) / duration (s)	1 071 / 4
Cliff after (GiB written / s)	3.8 / 4
Post-cliff steady sequential write (MiB/s)	63
Steady / burst	0.059
Fill: GiB, seconds, average MiB/s	4.3, 15, 295
Random 4 KiB write, QD32: phase average IOPS	30 090
per-second IOPS quartiles; median per-second p99 (µs)	14 306–15 343; 35 726
Recovery after 60 s idle: first / last 5 s (MiB/s)	1 104 / 1 104
NVMe temperature min–max (°C)	31.9–40.9

hypothesis (i)), rests on 11 s but matches the 62 MiB/s median of the original run's 1226 s post-cliff fill. That run never settled: it alternated between 60 MiB/s to 70 MiB/s (55 % of seconds) and 10 MiB/s to 20 MiB/s (32 %), averaging 47 MiB/s and drifting down. We take ∼60 MiB/s as the direct-to-QLC rate while folding; the long-run steady state is only partly tested.

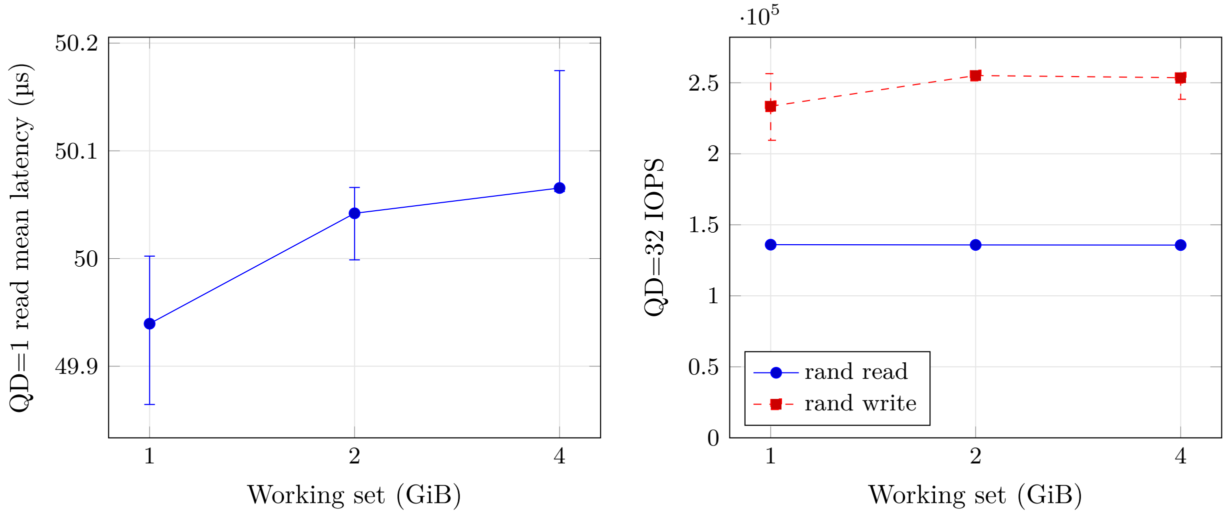
How large is the cache? We expected ∼20 GiB, but even on a trimmed, idle drive the cliff came at 3.8 GiB, only 0.7 GiB more than the original run's 3.1 GiB, so run order was not the main cause. This is the capacity available *in this drive state*; a plausible, untested explanation is a dynamic cache sized by drive-wide free space (unknown here, as only the scratch partition was trimmed), common in QLC client drives [3], [6].

The random-write phase (4 KiB, QD32, over the data just written) ran for eight full seconds at 12 471 to 16 548 IOPS (median 14 666) with a median per-second p99 of 36.0 ms, about 16× below the in-cache rate (235 830 IOPS). This supports hypothesis (ii) only in part: eight seconds cannot establish a steady state. In a final 0.7 s the rate jumped to ∼205k IOPS, as if SLC blocks had just been freed, and hit the 1 GiB cap, so the byte-weighted average in Table 9 (30 090 IOPS) mixes two states.

After the 60 s idle gap the recovery probe wrote its whole 1 GiB at 1 104 MiB/s, the burst rate, so hypothesis (iii) holds at this scale. In the original run the same gap after tens of gigabytes gave no recovery (61 then 19 MiB/s): recovery depends on the fold backlog. This is short-term behaviour, not the full-drive SNIA steady state [5].

Temperature. The composite temperature stayed at 31.9–40.9 °C here and 31.9 °C to 61.9 °C over both runs, far below the 86.85 °C warning threshold. Throughput fell at 35.9 °C and stayed low between 34.9 and 37.9 °C, so the cliff is the cache, not thermal throttling.

9.2 Working set (LBA range)



(a) QD=1 random read latency.

(b) QD=32 random read and write IOPS.

Figure 7: 4 KiB random I/O restricted to the first 1, 2 and 4 GiB of a freshly preconditioned 4 GiB test file (medians of 4 repetitions, 2 in each of a forward and a reverse pass, with IQR bars; data: lean re-run of 2026-10-21). No span effect is visible in this range.

QD=1 random-read latency is flat (49.9, 50.0 and 50.1 µs), QD=32 read IOPS differ by less than

0.3 %, and QD=32 writes stay at the in-cache rate (233 476–253 564 IOPS). Forward and reverse passes agree within 1 %, so there was no drift. At 4 bytes of map per 4 KiB page, 4 GiB needs only 4 MiB of map, so this says nothing about hypothesis (iv)'s larger spans. The original 1–60 GiB sweep showed QD=1 latency rising 37 %, but it ran right after the 60 GiB fill, in one direction, with its 1 GiB point already at 12 100 QD=1 IOPS against 20 000 here, so it cannot be separated from the fold backlog. Hypothesis (iv) remains untested beyond 4 GiB.

9.3 Data layout: why preconditioning order matters

Table 10: The same reads on the same 2 GiB file after a sequential fill and after a random 4 KiB overwrite (medians of 5 repetitions).

Read	after sequential fill	after random overwrite
128 KiB seq, QD=1 (MiB/s)	1 443	224
128 KiB seq, QD=32 (MiB/s)	2 093	229
4 KiB rand, QD=1 (IOPS)	18 803	18 747

An SSD places data in the order it was *written*, not by logical address, and Table 10 confirms hypothesis (v) strongly. After the random overwrite, QD=1 sequential reads drop from 1 443 to 224 MiB/s (6.4×), while 4 KiB random reads are unaffected. Our sub-prediction that QD=32 would hide the gap was *wrong*: it grows to 9.2×. Each fragmented 128 KiB request becomes up to 32 scattered page reads, and the drive completes ∼1830 such requests per second (about 58k page reads/s) at either depth, well below the 130k random reads/s it sustains at QD=32; why is untested. A sequential benchmark is meaningful only if the data's write history is controlled, which is why `run.sh` rewrites the file before every sequential-read group.

10 Misleading Setup

Hypothesis (from first principles, then compared with the data)

Dropping `O_DIRECT` lets the OS page cache serve repeated reads of the same region from RAM after the first pass, so a buffered benchmark will report throughput and latency that describe DRAM, not the SSD—orders of magnitude "better" than the real device, and entirely fictitious as a storage result.

Pitfall	Buffered I/O measuring the page cache instead of the device
Wrong setup	4 KiB sequential reads over a 64 MiB region, *no* `O_DIRECT`
Right setup	Identical workload *with* `O_DIRECT` (page cache bypassed)
Fixed	QD=1, 5 repetitions of 4 s; test file rewritten sequentially just before
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

10.1 Explanation

The buffered run reports 1 334 404 IOPS at 5 213 MiB/s and sub-microsecond latency, a 19× inflation over the correct 70 920 IOPS. None of it is a storage measurement: after the first pass the 64 MiB region lives in the page cache, so every later read is a `memcpy` from RAM. Omitting `O_DIRECT` is the default behaviour of `open()`, so an AI-generated benchmark falls into this readily. Two cheap detectors flag it in our data, and a third is expected to:

Table 11: Same 4 KiB sequential-read workload, direct vs. buffered: medians of 5 repetitions with the IQR in brackets. Only the `O_DIRECT` flag differs.

Metric	`O_DIRECT` (correct)	buffered (page cache)
IOPS	70 920 [68 204, 71 049]	1 334 404 [1 334 295, 1 334 424]
MiB/s	277 [266, 278]	5 213 [5 212, 5 213]
Mean latency (µs)	14.03 [14.00, 14.59]	0.68 [0.68, 0.68]
p99 latency (µs)	18.1 [18.1, 18.2]	1.15 [1.15, 1.15]

1. The throughput exceeds the 3756 MiB/s PCIe 3.0 ×4 link ceiling (Section 2), which no device result can do.
2. Sub-microsecond "storage" latency is physically impossible for NAND flash.
3. *Untested:* it should collapse to device speed once the region exceeds free RAM (63 GiB here).

The methodology avoids the trap by construction: `O_DIRECT` is the default for every real run, and `make test` asserts that buffered re-reads are more than 5× faster than direct ones. Related traps are a wrong effective queue depth, caught by L/QD (Table 8), and wrong-order preconditioning (Section 9.3).

11 Anomalies and Limitations

11.1 Unexpected results

- **The SLC cache is small** (3.8 GiB on a trimmed, idle drive, not ∼20 GiB), and its recovery after an idle gap depends on the fold backlog (Section 9).
- **Random writes beat random reads until the cache fills** (253 874 vs. 102 514 IOPS at QD=16), and the write collapses in the original sweeps were drive state (Sections 5 and 7).
- **Deep queues did not hide fragmentation** (9.2× at QD=32; Section 9.3), and **recent writes** halved read IOPS and raised p99.9/p50 to 24× (Section 8).

11.2 Limitations

- **Two runs are combined.** Only the confounded experiments were re-measured on an idle drive; everything else comes from `20261021-103201`, where the idle drive was about 5 % (QD=32) to 22 % (QD=8) faster for 4 KiB reads, so original-run read numbers may be slightly pessimistic.
- **Short lean fill.** 4.3 GiB, with 11 s post-cliff, 8.7 s of random writes and a 1 GiB recovery cap, so the steady state of hypothesis (ii) was not reached and the cache size was measured in one drive state.
- **Working set only to** 4 GiB on a clean drive; **closed-loop load only** (an open-loop generator would show delay against offered load).
- **No `iostat`/`perf`;** L/QD bounds device-queue occupancy from above.

- **SMART only after the run** (root needed): zero thermal events, media errors and over-temperature time, 100 % spare, but 28 % of rated life used, 72.61 TB written, 20 028 power-on hours and 2105 uninspected error-log entries. Wear may affect the cache size and tails.
- **Files on ext4, not the raw device** (at most 60 GiB, about 13 % of the drive); one drive, firmware and incompressible pattern.
- **Write points** use byte caps and 3–5 repetitions (∼15 % drive-state uncertainty); FIO, not the primary engine, agrees within 2 % on reads.

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5.5) for coding, debugging, and analysing the data: the initial harness up to 2026-10-20, then the 2026-10-21 rework and the analysis of the final runs. GCC gcc (GCC) 16.2.1 20260810 built the code. The AI wrote the code. It also produced a preliminary draft of the report, which I discarded; I wrote this report myself and verified all AI output. I executed the timed runs in a terminal on the machine in Table 1, using the scripts Claude Code wrote (`run.sh all`, then `SUSTAIN_GIB=60 run.sh device_state`, and later `rerun_clean.sh lean` after `sudo fstrim -v /`). Claude Code ran no timed benchmark; every number comes from those runs.

What they helped generate

`blkbench.c` (the O_DIRECT `libaio` benchmark), the data-safety gate (`safety.sh`) and its tests (`validate.sh`), the experiment driver (`run.sh`) with its CSV reduction, and the FIO job files and converter.

What was independently modified, tested, or rejected

Concrete corrections:

1. *The wrong device was being tested.* The scratch directory defaulted to the in-repo `data/scratch/`, which lives on the 2 TB SATA *HDD*, so the first full dataset showed millisecond QD=1 latencies. The scratch path was made overridable (`ACS_SCRATCH`) and pointed at the NVMe SSD. The AI's claim that "the NVMe is read-only to us" was false, as a one-line write test showed. The device check in `env.txt` was itself buggy: it stripped trailing digits from `nvme0n1p3` and printed `rotational: ?`. It now resolves the partition to its parent disk.
2. An early version measured *submission* latency rather than *completion* latency; it now times each I/O to completion.
3. A convenience default left the page cache enabled and reported ∼2 million IOPS. That case became the required pitfall (Section 10), and `O_DIRECT` became the default.
4. The report claimed a `0xA5` write fill, but the timed write buffers were never initialised. They are now filled once with an incompressible xorshift pattern, and `make test` checks that it does not compress.
5. The report claimed `validate.sh` refused `/dev/nvme0n1` when it only tested `/dev/sda`; it now tests both. It also claimed a direct-vs-buffered assertion that did not exist; the assertion has been added.

6. An AI-drafted sentence said p99 “rises monotonically with write fraction”. The table contradicts it (pure write is below 50/50), and the sentence was corrected.
7. The first version of the sustained-write summary defined “burst” as the median of the first 10 s. On the real data, where the burst lasted only 3 s, that reported a misleading 66 MiB/s burst and a spurious cliff at 11 GiB. The definitions were fixed and the summary was regenerated from the unchanged raw log (`run.sh reprocess`).
8. The earlier report attributed the small (3.1 GiB) cache to the run order. The lean re-run on a trimmed, idle drive found 3.8 GiB, so that explanation was withdrawn (Section 9). The summary column `rand_steady_iops` is a byte-weighted phase average, and in the lean run a final 0.7 s at SLC speed doubles it. Checked against the per-second log, the text now quotes the per-second median of the eight full seconds as the steady value.
9. Claims carried over from the pilot data were re-checked against the final run and corrected where the data disagreed: the QD 1→4 scaling, the 26 % gain from QD 32 to 64 (15 %), the constant p99.9/p50 ratio, and the prediction that deep queues hide fragmentation.

Validation

`make test` checks the safety gate (both block devices, a symlink to one, `/etc/passwd`, `/`), the incompressible fill, physical orderings, the $L \leq$ QD invariant, page-cache bypass and the FIO converter. FIO cross-checks `blkbench` (Section 3.1). Every figure traces to a committed raw/processed file (Section A).

13 Conclusion

At QD=1 the Micron 2210’s write buffer makes 4 KiB random writes faster than reads. Block size moves the bottleneck from per-request latency to bandwidth, but even at QD=32 sequential reads reach only 57 % of the PCIe link, so the NAND back end is the ceiling, and writes saturate the ∼1.06 GiB/s SLC ingest rate even at QD=1. Mixing reads and writes is worse than either pure workload. The read knee lies at QD 64 under an explicit 10 %-per-doubling rule; beyond it latency doubles per doubling of QD, and L/QD of 0.987–0.998 confirms that the saturation is the device’s, not the harness’s.

Device state matters as much as configuration. Even on a trimmed, idle drive the SLC cache held only 3.8 GiB, plausibly because it scales with free space, and past it writes fell 17×. Recent writes halved read IOPS and stretched p99.9/p50 from 5.7× to 24×, and the write history of the data made “sequential” reads 6–9× slower. Temperature stayed far below the warning threshold.

The sharpest lessons are methodological. Without `O_DIRECT` the same workload reported page-cache fiction, and the first dataset was taken on the HDD until the `rotational` flag gave it away. A storage measurement can be trusted only if it bypasses the cache, verifies its effective queue depth, controls the data’s write history and the drive’s recent workload, and confirms which device it is talking to.

References

[1] J. Axboe, *Fio: Flexible I/O tester*, Version 3.42 on the test host; used to cross-check the libaio benchmark (blkbench). [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

[3] Micron Technology, Inc., *Micron 2210 QLC SSD with NVMe: Product brief*, Rev. A, Micron Technology, Apr. 2020. [Online]. Available: `https://micron.cn/content/dam/micron/global/public/products/product-flyer/2210-qlc-ssd-product-brief.pdf`

[4] PCI-SIG, *PCI Express base specification, revision 3.0*, PCI-SIG, Nov. 2010.

[5] Storage Networking Industry Association, *Solid state storage (SSS) performance test specification (PTS)*, version 2.0.2, SNIA, Oct. 2020. [Online]. Available: `https://www.snia.org/tech_activities/standards/curr_standards/pts`

[6] B. Tallis, *The Intel SSD 660p SSD review: QLC NAND arrives for consumer SSDs*, AnandTech, Aug. 7, 2018. [Online]. Available: `https://www.anandtech.com/show/13078/the-intel-ssd-660p-ssd-review-qlc-nand-arrives`

A Reproducibility

Code and data live in `ACS-Code/`, the reports in `ACS-Latex/`, at commit `e33345f7874f`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout e33345f7874f
cd ACS-Code
make tools                                     # build/tools/summarize
make -C SSD_Performance_Profiling              # build/blkbench (O_DIRECT, libaio)
make -C SSD_Performance_Profiling test         # safety + sanity checks (~70 MiB written, deleted)
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
export ACS_SCRATCH="$HOME/acs_ssd_scratch"     # a directory on the SSD under test
SSD_Performance_Profiling/scripts/run.sh dryrun         # device, link, temperature, space; writes
    nothing
SSD_Performance_Profiling/scripts/run.sh all            # 2 GiB-file sweeps + fio_check (~23 min
    measured)
SUSTAIN_GIB=60 SSD_Performance_Profiling/scripts/run.sh device_state  # sustained + working set (
    needs 70 GiB free)
scripts/sync_data.sh SSD_Performance_Profiling
sudo fstrim -v /                               # lean re-run: 21:34:54, then idle >= 30 min
SSD_Performance_Profiling/scripts/rerun_clean.sh lean    # 20261021-220614, 22:06-22:32 EDT
scripts/sync_data.sh SSD_Performance_Profiling
cd ../ACS-Latex && make ssd
```

`rerun_clean.sh` (Section 2 and Table 3) first checks free space, drive idleness and recent-TRIM evidence; its full, non-lean mode has not been run. The lean run's console output is in `data/raw/rerun_clean_20261021-220614.log`, and the later root SMART log in `data/raw/smart/20261021-2335_post_smart_log.txt`. Without `ACS_SCRATCH`, the in-repo `data/scratch/` (an HDD here) is profiled; the safety gate applies either way. Knobs (`REPS`, `RUNTIME`, `WRITE_CAP_MIB`, `SUSTAIN_GIB`, `ENGINE=fio`, . . .) are documented in `run.sh`.

Each raw directory holds `raw.csv` and `env.txt`; `tools/summarize` reduces them to the CSVs in `data/`, from which every table and quoted number is read at compile time.

Table 12: Raw data behind each figure (`ACS-Code/SSD_Performance_Profiling/data/raw/...`). Where two directories are listed, the lean (`_lean`) rows replace the matching configurations of the original run.

Figure / table	Raw run directory
Table 6	`zero_queue/20261021-103201`
Figures 1 and 2	`block_size/20261021-103201` + `block_size_lean/20261021-220614`
Figure 3, Table 7	`rw_mix/20261021-103201`
Figure 4, Table 8	`queue_depth/20261021-103201` + `queue_depth_lean/20261021-220614`
Figure 5	`tail/20261021-220614` (original run: `tail/20261021-103201`)
Table 11	`pitfall/20261021-103201`
Table 10	`layout/20261021-103201`
Tables 4 and 5	`fio_check/20261021-103201` + `fio_check_lean/20261021-220614` (FIO JSON under `fio/`)
Figure 6, Table 9	`sustained/20261021-220614` (original run: `sustained/20261021-105439`)
Figure 7	`working_set/20261021-220614` (original run: `working_set/20261021-105439`)
Table 3	`rerun_clean_20261021-220614.log`
tmpfs L/QD (Section 7)	`validate_tmpfs/20261021-231433.txt` (`make test` output, scratch on `/dev/shm`)